

UNIVERSIDADE DE AVEIRO

Departamento de Eletrónica, Telecomunicações e Informática

Redes e Sistemas Autónomos

Cooperative Lane Merge

via Comunicação V2V

Sistema autónomo de apoio à inserção cooperativa em faixa de rodagem

Pedro Melo n.º 114208

6 de junho de 2026

Repositório: [\(inserir URL\)](#)

Vídeo da demo: [\(inserir URL / Discord\)](#)

Conteúdo

1	Introdução	2
2	Objetivos	2
2.1	Problema	2
2.2	Objetivo geral	3
2.3	Objetivos específicos	3
2.4	Síntese	3
3	Arquitetura	4
3.1	Visão geral em camadas	5
3.2	Nós e topologia	6
3.3	Mapa de tópicos Zenoh	6
3.4	Modelo de papéis	7
3.5	Tipos de mensagens trocadas	7
4	Implementação	8
4.1	Stack e fluxo de uma mensagem	8
4.2	Beaconing e NeighbourTable	8
4.3	Geometria	9
4.4	Modelo cinemático	9
4.5	Construtores de MCM	9
4.6	O protocolo MergeProtocol	9
4.6.1	Lado do Merge Car (rampa)	9
4.6.2	Lado do veículo da estrada principal	10
4.7	Algoritmos de decisão e pseudo-código	10
4.8	Diagrama de sequência do protocolo	10
4.9	Orquestração e visualização	11
4.10	Modo Demo	11
5	Fluxo da Demonstração	11
5.1	Preparação	11
5.2	Timeline	11
5.3	Aleatoriedade e variáveis	12
5.4	Cenários	12
6	Resultados	12
7	Conclusão	12

1 Introdução

O aumento da densidade de tráfego nas entradas de autoestrada torna a inserção de veículos provenientes de rampas de acesso um dos pontos mais críticos para a fluidez e a segurança rodoviária. Tradicionalmente, esta manobra depende do julgamento individual de cada condutor, o que frequentemente origina travagens bruscas, paragens na rampa ou inserções forçadas que propagam ondas de congestionamento para a via principal.

Este projeto, desenvolvido no âmbito da unidade curricular de Redes e Sistemas Autónomos, explora como a comunicação Veículo-a-Veículo (V2V) pode substituir esse julgamento individual por uma **negociação cooperativa e distribuída**, sem qualquer entidade central de controlo. O cenário estudado é o de um veículo — o *Merge Car* (MC) — que se aproxima da via principal a partir de uma rampa de acesso e necessita de coordenar a sua inserção com os veículos que já circulam nessa via. Em vez de impor a sua entrada, o MC inicia um protocolo de coordenação de manobra em que os veículos afetados são identificados em tempo real, decidem cooperativamente um abrandamento controlado e concedem ao MC uma janela temporal segura para a inserção.

A solução assenta integralmente em mensagens normalizadas ETSI C-ITS: *Cooperative Awareness Messages* (CAM) para a consciência situacional periódica e *Manoeuvre Coordination Messages* (MCM) para a negociação da manobra. A pilha de comunicação é fornecida pelo Vanetza-NAP, o transporte de mensagens pelo middleware Zenoh, e toda a lógica dos veículos corre em containers Docker independentes, garantindo que cada veículo é um nó autónomo que só conhece a sua vizinhança através das CAMs que recebe. Uma dashboard web em tempo real permite observar a negociação e a manobra à medida que decorrem.

Este relatório descreve os objetivos do trabalho (Secção 2), a arquitetura do sistema (Secção 3), a implementação detalhada do protocolo e dos algoritmos de decisão (Secção 4), o fluxo da demonstração (Secção 5) e os resultados obtidos nos vários cenários testados (Secção 6), incluindo casos com dois veículos a inserir-se em simultâneo.

2 Objetivos

2.1 Problema

A inserção de um veículo proveniente de uma rampa de acesso na via principal é uma das manobras com maior potencial de conflito da circulação rodoviária. Sem cooperação entre veículos, o condutor que vem da rampa só dispõe da sua própria perceção visual para encontrar uma folga: quando esta não existe, vê-se obrigado a forçar a entrada, a travar bruscamente ou mesmo a parar no final da rampa à espera de espaço. Cada uma destas reações tem custos:

- **Inserção forçada** — obriga os veículos da via principal a travagens de emergência, comprometendo a segurança;
- **Paragem na rampa** — elimina a velocidade de aproximação necessária para uma inserção suave e cria risco de colisão traseira na própria rampa;
- **Travagens reativas** — propagam-se para montante sob a forma de ondas de choque,

degradando a fluidez de todo o troço.

A raiz do problema é a *falta de informação antecipada e de coordenação*: os veículos da via principal só percebem a intenção de inserção quando o veículo da rampa já está praticamente sobre o ponto de conflito, deixando pouca margem para uma resposta suave.

2.2 Objetivo geral

O objetivo central deste trabalho é demonstrar que a comunicação Veículo-a-Veículo (V2V) permite resolver este problema através de uma **negociação cooperativa, distribuída e antecipada**, sem qualquer entidade central de controlo e assente exclusivamente em mensagens normalizadas ETSI C-ITS (CAM e MCM). Pretende-se que o veículo da rampa — o *Merge Car* (MC) — anuncie a sua intenção com antecedência suficiente para que os veículos afetados se ajustem de forma coordenada e lhe abram uma janela temporal segura, substituindo a reação tardia por uma manobra planeada.

2.3 Objetivos específicos

Para concretizar o objetivo geral, definiram-se os seguintes objetivos específicos:

- O1. Consciência situacional contínua** — cada veículo difunde periodicamente a sua posição, velocidade, aceleração e dimensões via CAM, construindo uma visão local do tráfego sem qualquer configuração prévia da vizinhança.
- O2. Protocolo de negociação de manobra** — definir e implementar um protocolo distribuído sobre MCM que permita ao MC pedir autorização e aos restantes veículos responder cooperativamente.
- O3. Detecção de conflito** — determinar, em cada veículo da via principal e em tempo real, se irá efetivamente partilhar a zona de conflito com o MC, combinando a geometria da estrada com uma janela temporal de ocupação.
- O4. Abrandamento coordenado em cadeia** — quando há conflito, calcular uma velocidade-alvo viável e propagar o pedido de abrandamento aos veículos a montante, de modo a abrir a folga necessária sem travagens bruscas.
- O5. Garantia de segurança** — assegurar que a manobra nunca compromete a segurança: se a negociação não concluir a tempo, o MC recorre a um mecanismo de *stop-and-wait* no final da rampa e volta a tentar.
- O6. Suporte a múltiplos veículos a inserir-se** — estender o protocolo a cenários com dois veículos a fundir-se simultaneamente a partir da rampa.
- O7. Observabilidade** — disponibilizar uma visualização em tempo real da simulação e da troca de mensagens, para validação e demonstração do protocolo.

2.4 Síntese

A Tabela 1 resume como cada objetivo específico é concretizado na implementação, remetendo para a secção correspondente.

Objetivo	Mecanismo de concretização	Secção
O1 – Consciência situacional	Beaconing CAM a 100 ms + <code>NeighbourTable</code> com janela de 200 ms	4
O2 – Protocolo de negociação	6 subtipos de MCM (<code>MERGE_REQUEST</code> , <code>SLOW-DOWN_REQUEST/GRANT</code> , <code>MERGE_GRANT</code> , <code>MERGE_CONFIRMED</code> , <code>EXECUTION_STATUS</code>)	4
O3 – Detecção de conflito	Projeção geométrica (<code>project_t</code>) + sobreposição temporal entre a ocupação do MC e a entrada/saída do veículo na zona	4
O4 – Abrandamento em cadeia	Velocidade-alvo por busca binária (<code>_solve_target_speed</code>) e propagação de <code>SLOW-DOWN_REQUEST</code> ao veículo de trás	4
O5 – Garantia de segurança	Verificação cinemática de travagem (<code>can_brake_in_time</code>) e paragem em <code>stop_t</code> com retentativa	4
O6 – Múltiplos MC	Mesmo protocolo na rampa: o MC propaga <code>SLOW-DOWN_REQUEST</code> ao veículo atrás na rampa	4
O7 – Observabilidade	Ponte Zenoh→WebSocket (<code>bridge.py</code>) e dashboard HTML em tempo real	4

Tabela 1: Mapeamento dos objetivos específicos para os mecanismos de implementação.

3 Arquitetura

O sistema está organizado em quatro camadas com responsabilidades bem separadas, tal como ilustra a Figura 1. Esta secção fixa o vocabulário (nós, papéis, tópicos e mensagens) que a Secção 4 utiliza para detalhar a implementação.

3.1 Visão geral em camadas

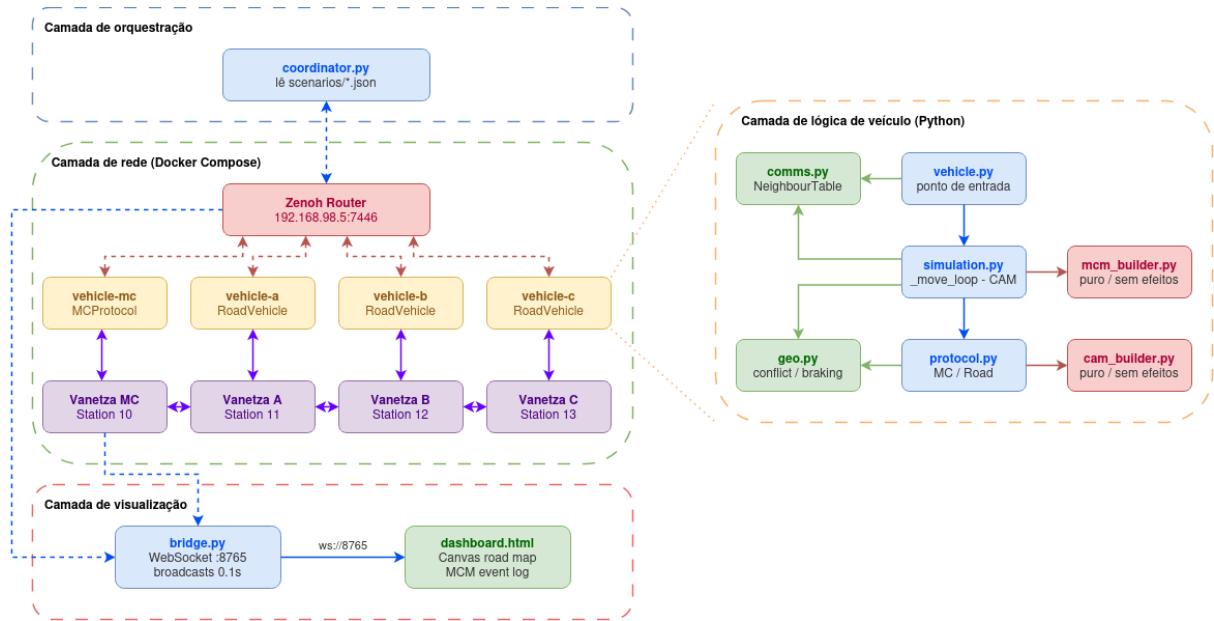


Figura 1: Arquitetura em camadas do sistema.

Camada de orquestração. Um único processo, o `coordinator.py` (GUI Tkinter), lê os ficheiros de cenário `cenarios/*.json`, publica o cenário escolhido no router Zenoh e aguarda que todos os veículos ativos reportem a conclusão. É o ponto de entrada da demonstração, mas *não* participa na negociação — apenas dá o pontapé de saída e recolhe o estado de cada veículo.

Camada de rede. Materializada por `docker-compose` sobre a rede Docker externa `vanetzalan0`. Contém o *router Zenoh* (canal do coordinator) e, por cada veículo, um par de containers: um container **Vanetza-NAP**, que implementa a pilha ETSI C-ITS (codificação/ decodificação ASN.1 e difusão pelo “ar”), e um container **Python**, que corre a lógica do veículo. É nesta camada que vivem as *stations* 10–13.

Camada de lógica de veículo (Python). É o cérebro de cada nó, organizado em módulos: `vehicle.py` (ponto de entrada), `simulation.py` (laço de movimento e emissão de CAM), `protocol.py` (a classe `MergeProtocol`, que concentra toda a negociação), apoiados por `geo.py` (geometria e modelo de travagem), `comms.py` (sessões Zenoh e `NeighbourTable`) e pelos construtores puros de mensagens `cam_builder.py` e `mcm_builder.py`. Esta camada é o objeto central da Secção 4.

Camada de visualização. O `bridge.py` subscreve o tráfego V2X (CAM e MCM) decodificado, mantém o estado de cada veículo e difunde, a cada 100 ms, uma trama JSON por WebSocket (porta 8765) para a `dashboard.html`, que desenha as estradas, os veículos e o registo de eventos MCM em tempo real.

Os dois planos de comunicação. A Figura 1 distingue, por cor, dois planos. O *plano V2X* (ligações entre cada veículo e o seu Vanetza e entre os Vanetza) transporta o tráfego C-ITS (CAM/MCM) e representa o meio rádio: na realidade, cada veículo difunde-o pelo broker Zenoh do seu *próprio* container Vanetza-NAP (`tcp/localhost:7447`) — não existe broker partilhado para o V2X, e a figura abstrai esses brokers como ligações diretas. O *plano de orquestração* (ligações ao bloco central “Zenoh Router”, `192.168.98.5:7446`) transporta apenas as mensagens de controlo da demonstração (cenário, *ready*, *done*). Assim, nenhum veículo comunica com outro por um canal partilhado privilegiado — toda a coordenação inter-veicular acontece exclusivamente através de mensagens C-ITS difundidas pelo respetivo Vanetza.

3.2 Nós e topologia

O cenário envolve **quatro veículos** — o Merge Car (MC) e os veículos da via principal A, B e C. Cada veículo é, na prática, um *par de containers* (Vanetza-NAP + Python) com uma identidade C-ITS fixa, resumida na Tabela 2. A estes junta-se o router Zenoh (canal de orquestração) e a cadeia de observabilidade — o `bridge.py` e a `dashboard` —, que apenas consome o tráfego decodificado para visualização, sem participar na negociação.

Veículo	Papel	Station ID	MAC	IP Vanetza
MC	Merge Car (rampa)	10	6e:06:e0:03:00:10	192.168.98.10
A	Via principal	11	6e:06:e0:03:00:11	192.168.98.11
B	Via principal	12	6e:06:e0:03:00:12	192.168.98.12
C	Via principal	13	6e:06:e0:03:00:13	192.168.98.13
Router Zenoh (coordinator)		—	—	192.168.98.5:7446

Tabela 2: Nós do sistema: identidade C-ITS e endereçamento. Cada veículo expõe ainda o broker Zenoh do seu Vanetza na porta 7447.

Cada container Python liga-se ao broker Zenoh do seu próprio Vanetza (`VANETZA_ZENOH_URL`) para o tráfego V2X e ao router central (`COORDINATOR_ZENOH_URL`) para o canal de orquestração. A geração nativa de CAM no Vanetza está **desativada** (`VANETZA_CAM_PERIODICITY=0`): toda a emissão de CAM é controlada pela camada Python, garantindo um *tick* de simulação coerente entre movimento e *beaconing*.

3.3 Mapa de tópicos Zenoh

A comunicação entre as camadas reduz-se a um pequeno conjunto de tópicos Zenoh (Tabela 3). Os tópicos `vanetza/in/*` e `vanetza/out/*` representam, respetivamente, a injeção de uma mensagem na pilha C-ITS e a sua receção após passar pelo “ar”; os tópicos `coordinator/*` constituem o canal de controlo da demonstração.

Tópico	Publica	Subscreve	Conteúdo
vanetza/in/cam	Veículo (Python)	Vanetza-NAP	CAM a difundir (dict JSON)
vanetza/out/cam	Vanetza-NAP	Veículos, bridge	CAM recebida do ar
vanetza/in/mcm	Veículo (Python)	Vanetza-NAP	MCM a difundir (dict JSON)
vanetza/out/mcm	Vanetza-NAP	Veículos, bridge	MCM recebida do ar
coordinator/scenario	coordinator	Veículos, bridge	Cenário completo (JSON)
coordinator/ready/<sid>	Veículo	coordinator	<i>Heartbeat</i> : veículo vivo
coordinator/done/<sid>	Veículo	coordinator	Veículo concluiu o cenário

Tabela 3: Mapa de tópicos Zenoh. <sid> é o *Station ID* do veículo.

3.4 Modelo de papéis

Uma decisão central da arquitetura é que **todos os veículos correm exatamente o mesmo código**. Não existe um binário “MC” e outro “veículo da estrada”: o papel *emerge em tempo de execução* do tipo de estrada onde o veículo é colocado pelo cenário. Um veículo posicionado numa estrada do tipo **ramp** com campo **merges_into** ativa o comportamento de Merge Car dentro do **MergeProtocol**; o mesmo código numa estrada **main** responde como veículo alvo da negociação. Esta unificação tem duas consequências importantes: (i) os cenários multi-MC são suportados sem código adicional — basta colocar dois veículos na rampa (ver cenários 04–06); e (ii) toda a lógica de negociação vive numa única classe, descrita em detalhe na Secção 4.

Coerentemente, **nenhum veículo tem conhecimento pré-configurado da sua vizinhança**. Quem está à frente, quem está atrás, quem entra em conflito e em que estrada cada um circula é inferido em tempo real a partir das posições GPS recebidas nas CAM e da geometria das estradas definida no cenário.

3.5 Tipos de mensagens trocadas

Apenas dois tipos de mensagem ETSI C-ITS são usados (Tabela 5). A **CAM** é o *beaconing* periódico que sustenta a consciência situacional; a **MCM** implementa o protocolo de negociação e desdobra-se em seis subtipos. Cada MCM transporta no **basicContainer** três campos que, em conjunto, identificam o subtipo: **mcmType** (tipo de mensagem), **itssRole** (papel do emissor) e **manoeuvreId** (identificador da sessão); o significado de cada valor consta da Tabela 4. Note-se em particular que o **manoeuvreId** — no standard, apenas um identificador de 1 byte — é usado, por convenção do projeto, para distinguir um **MERGE_GRANT** (0–127) de um **SLOWDOWN_GRANT** (128–255), que de outra forma seriam indistinguíveis.

Campo	Valor	Significado
mcmType	1	<i>request</i> — pedido de manobra
	2	<i>response</i> — resposta a um pedido
	7	<i>executionStatus</i> — estado de execução da manobra
itssRole	1	<i>coordinatingItss</i> — ITS-S que coordena o merge (o MC)
	3	<i>targetVehicle</i> — veículo-alvo (a quem se pede para abrandar)

Tabela 4: Significado dos códigos de `mcmType` e `itssRole` (enumerações ETSI) usados no `basicContainer`.

Subtipo	mcmType	itssRole	manoeuvreId	Direção	Propósito
MERGE_REQUEST	1	1	0–127	MC → todos	Pedir autorização; inclui zona de conflito e janela de ocupação
SLOWDOWN_REQUEST	1	3	0–127	em conflito → de trás	Propagar pedido de abrandamento na cadeia (via <code>executantID</code>)
SLOWDOWN_GRANT	2	3	128–255	atrás → à frente	Confirmar/recusar a aceitação do abrandamento
MERGE_GRANT	2	3	0–127	cada veículo → MC	Conceder a passagem ao MC, individualmente
MERGE_CONFIRMED	2	1	0–127	MC → todos	Confirmar (<i>agree</i>) ou abortar a manobra
EXECUTION_STATUS	7	1	0–127	MC → todos	MC já entrou na via; veículos retomam a velocidade

Tabela 5: CAM (*beaconing* periódico) e os seis subtipos de MCM do protocolo. O significado dos códigos de `mcmType`, `itssRole` e `manoeuvreId` consta da Tabela 4.

4 Implementação

[**TODO:** Secção central — nível de detalhe alto. Cada subsecção será preenchida com referências às funções reais, fórmulas e pseudo-código.]

4.1 Stack e fluxo de uma mensagem

[**TODO:** Python `build_cam/build_mcm` → `vanetza/in/*` → Vanetza-NAP (codec ASN.1/UPER) → `ar` → `vanetza/out/*` → `callbacks`. Escalas do codec: `lat/lon` $\times 10^7$, `speed` $\times 100$, `accel` $\times 10$; `sentinels`.]

4.2 Beaconing e NeighbourTable

[**TODO:** `build_cam` (campos ETSI, `sentinels` de *unavailable*); `make_cam_callback`; janela de 200 ms na `NeighbourTable`; vizinhança 100% derivada de CAMs.]

4.3 Geometria

[**TODO:** haversine, project_t (projeção paramétrica $t \in [0, 1]$), compute_bearing, road_length, is_on_road, find_vehicle_behind (≤ 200 m), conflict_zone_t, interpolação linear de movimento. Incluir fórmulas.]

4.4 Modelo cinemático

[**TODO:** advance_speed (± 4 m/s²), predict_motion, time_to_cover_distance, can_brake_in_time, merge_entry_margin, mc_stop_t. Fórmulas e justificação das margens.]

4.5 Construtores de MCM

[**TODO:** Funções puras de mcm_builder.py: estrutura de cada subtipo (basicContainer, vehicleManoeuvreContainer, TRR/waypoints, manoeuvreAdvice, temporalCharacteristics, responseContainer); endereçamento via executantID; intervalos de manoeuvreId (MERGE_GRANT 0–127 vs SLOWDOWN_GRANT 128–255).]

4.6 O protocolo MergeProtocol

4.6.1 Lado do Merge Car (rampa)

[**TODO:** _tick_ramp, cálculo do ETA e horizonte de 4 s, _maybe_send_merge_request (janela de ocupação), recolha de grants (grants_received), _on_all_granted $\rightarrow$ MERGE_CONFIRMED, retry/cooldown, paragem em stop_t, slowdown na rampa (multi-MC). Referir Figura 2.]

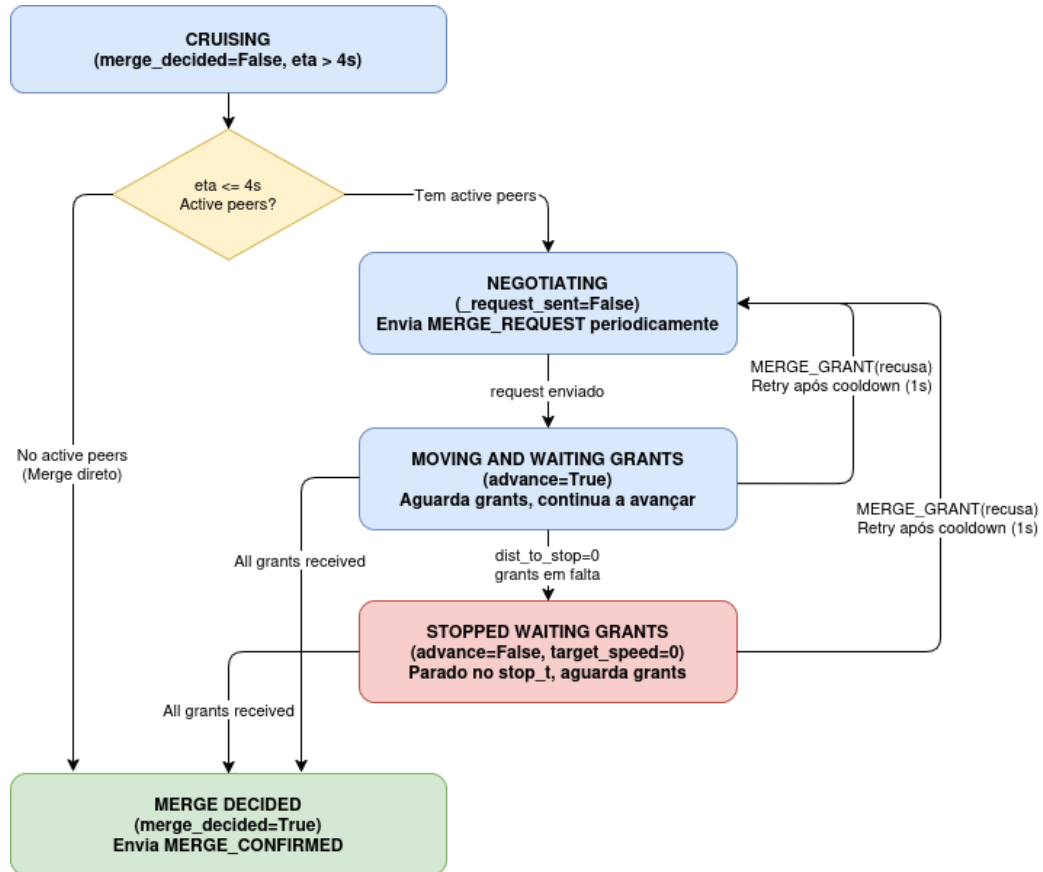


Figura 2: Máquina de estados do Merge Car.

4.6.2 Lado do veículo da estrada principal

[**TODO:** `_on_merge_request` (detecção de conflito), cálculo da velocidade-alvo, propagação `SLOWDOWN_REQUEST` / `_on_slowdown_request`, fim-de-cadeia + `_on_slowdown_grant`, envio de `MERGE_GRANT`, aplicação do abrandamento em `MERGE_CONFIRMED`, retoma em `EXECUTION_STATUS`. Referir Figura 3.]

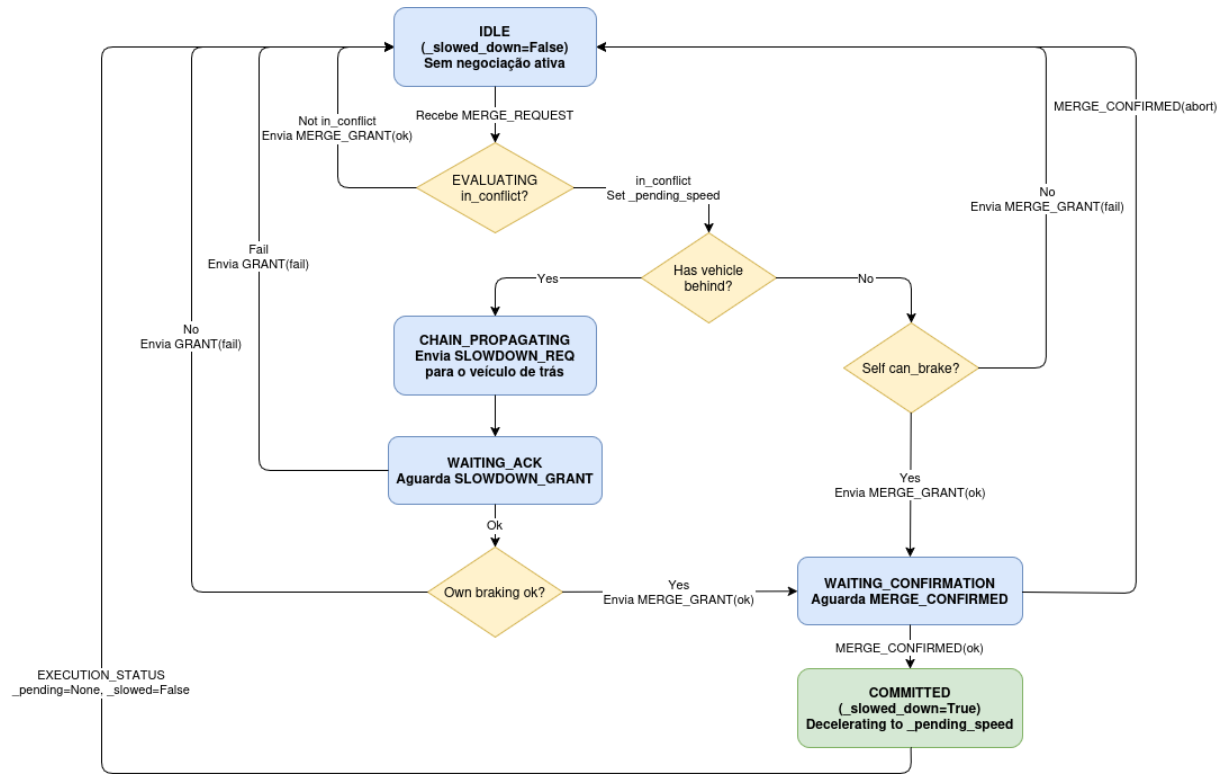


Figura 3: Máquina de estados do veículo na via principal.

4.7 Algoritmos de decisão e pseudo-código

[**TODO:** (a) Detecção de conflito (overlap temporal); (b) busca binária da velocidade-alvo `_solve_target_speed`; (c) propagação em cadeia do abrandamento. Um ambiente `algorithm` por algoritmo.]

4.8 Diagrama de sequência do protocolo

[**TODO:** Explicação passo-a-passo com base na Figura 4: `MERGE_REQUEST` → `MERGE_GRANT` directos → `SLOWDOWN_REQUEST`/`GRANT` em cadeia → `MERGE_GRANT` → `MERGE_CONFIRMED` → `EXECUTION_STATUS`; ramo alternativo `ABORT`.]

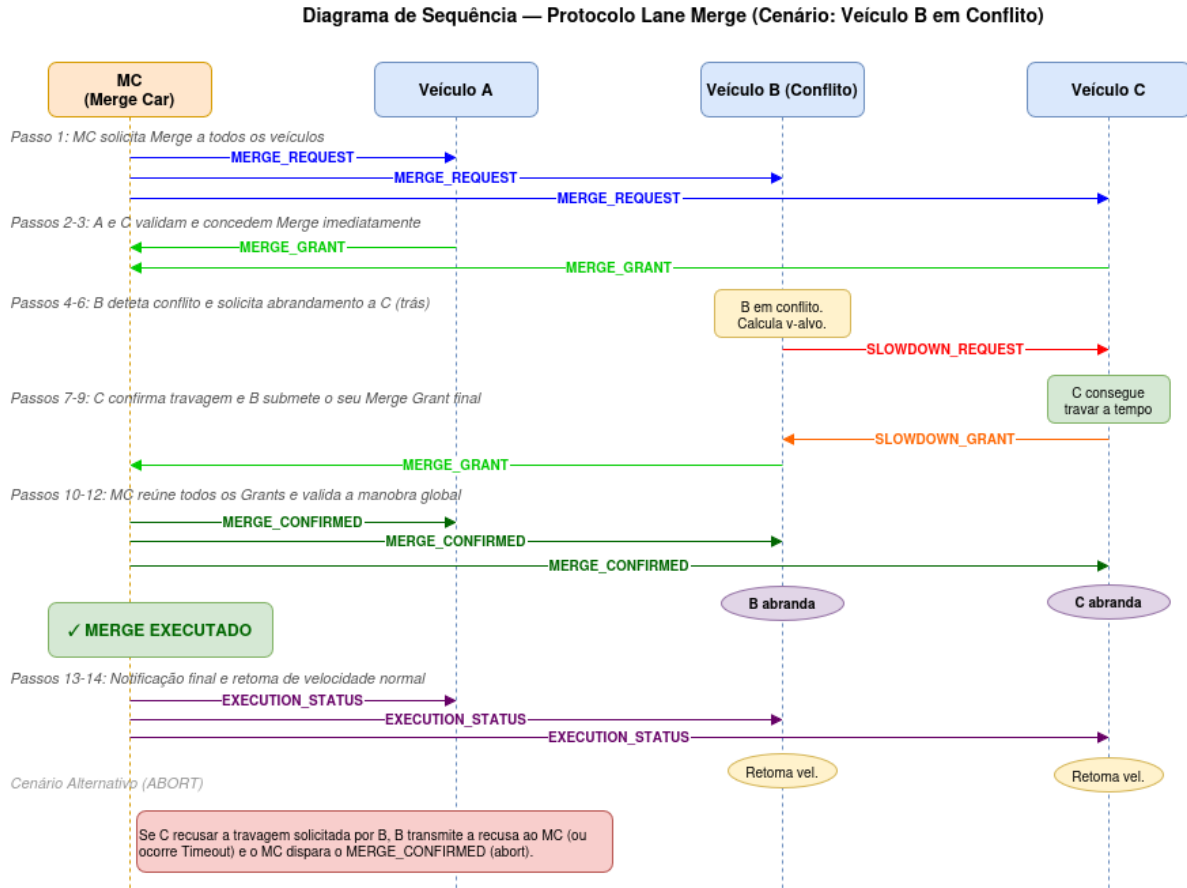


Figura 4: Diagrama de sequência do protocolo (cenário com o veículo B em conflito).

4.9 Orquestração e visualização

[**TODO:** coordinator.py (GUI Tkinter, dispatch de cenários via Zenoh, ready/done, Demo Mode, Draw Exag); bridge.py (Zenoh→WebSocket, classificação de MCM, estados visuais, merge_point por interseção geométrica); dashboard.html; run.sh.]

4.10 Modo Demo

[**TODO:** DemoMergeProtocol: pausas configuráveis antes de processar cada MCM relevante para tornar a negociação observável passo a passo.]

5 Fluxo da Demonstração

5.1 Preparação

[**TODO:** run.sh; escolher cenário no coordinator; dashboard aberta.]

5.2 Timeline

[**TODO:** Ponto de partida → MERGE_REQUEST ao atingir o horizonte de 4 s → decisões em função das mensagens recebidas → abrandamento/merge → retoma de velocidade. Como termina a demo.]

5.3 Aleatoriedade e variáveis

[**TODO:** Aleatório: manoeuvreId do SLOWDOWN_GRANT (128–255), jitter de rede e ordem de chegada de CAMs/MCMs. Controlável: cenário (posições iniciais), Demo Mode + pausa, Draw Exag, speed limits e comprimentos no JSON.]

5.4 Cenários

[**TODO:** Tabela dos 6 cenários: 01 standard; 02 colisão B; 03 colisão A+B+C; 04 2-MC standard; 05 2-MC colisão B; 06 2-MC colisão todos — com o resultado esperado de cada.]

6 Resultados

[**TODO:** Por cenário: comportamento observado (conflito, cadeia de abrandamento, merge sem colisão) com evidência (screenshots da dashboard + logs anotados). Validação dos casos de colisão (sem protocolo colidiria; com protocolo é seguro). Caso de abort + retry. Caso fallback stop-and-wait. Multi-MC (cadeia também na rampa). Discussão: limitações e simplificações do modelo.]

7 Conclusão

[**TODO:** Síntese do alcançado vs objetivos; lições aprendidas; trabalho futuro.]

Referências